FATEC IPIRANGA

ED

ESTRUTURA DE DADOS

PONTEIRO

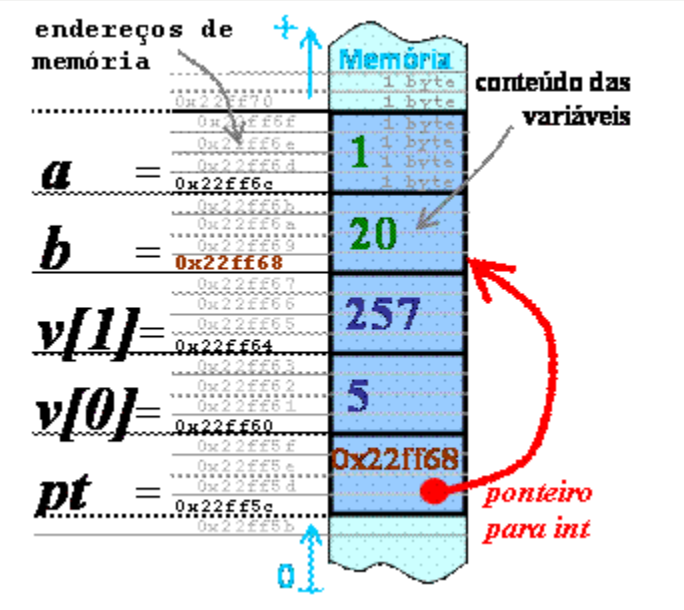
17/10/2025

Prof. Carlos Veríssimo

PONTEIROS

INTRODUÇÃO

- O objetivo deste documento é abordar o conceito de ponteiro, com uma robusta prática de laboratório, na linguagem C



PONTEIROS

CONCEITUANDO

- Um ponteiro **é uma variável** que armazena o **endereço de memória** de outra variável.
- Em vez de armazenar um valor direto, ele aponta para a localização onde o valor está armazenado.

PONTEIROS

DECLARANDO PONTEIROS

Para declarar um ponteiro utilizamos o **operador *** antes do nome da variável.

```
1
2 //Declaração de ponteiro
3 int *ptr;
```

Declaramos **ptr:** é um ponteiro para um inteiro.

PONTEIROS

OPERADOR DE REFERÊNCIA: &

Objetivo: Obter o endereço de uma variável.

```c
#include <stdio.h>

int main()
{

    int var = 10;
int *ptr = &var; // ptr agora contém o endereço de var
printf("Valor de var: %d\n", *ptr);
return 0;

}
```

```
Valor de var: 10


...Program finished with exit code 0
Press ENTER to exit console.
```

PONTEIROS

ACESSO AO VALOR

Para acessar o **valor armazenado no endereço** apontado pelo ponteiro, utilizamos *

```c
#include <stdio.h>

int main()
{
int var = 10;
int *ptr = &var;
int value = *ptr; // value agora é 10

printf("O valor value é: %d ",value);

    return 0;
}
```

```
O valor value é: 10

...Program finished with exit code 0
Press ENTER to exit console.
```

PONTEIROS

MANIPULAÇÕES BÁSICAS

```c
#include <stdio.h>

int main() {
    int var = 20;
    int *ptr = &var;

    printf("Valor de var: %d\n", var);
    printf("Endereço de var: %p\n", &var);
    printf("Valor armazenado em ptr: %p\n", ptr);
    printf("Valor apontado por ptr: %d\n", *ptr);

    return 0;
}
```

```
Valor de var: 20
Endereço de var: 0x7ffea168c0ac
Valor armazenado em ptr: 0x7ffea168c0ac
Valor apontado por ptr: 20


...Program finished with exit code 0
```

PONTEIROS

ITERANDO ARRAY

```c
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int *ptr = arr; // ptr aponta para o primeiro elemento do array

    for (int i = 0; i < 5; i++) {
        printf("Valor de arr[%d]: %d\n", i, *(ptr + i));
    }

    return 0;
}
```

```
Valor de arr[0]: 10
Valor de arr[1]: 20
Valor de arr[2]: 30
Valor de arr[3]: 40
Valor de arr[4]: 50
```

PONTEIROS

PONTEIROS EM FUNÇÕES

```c
#include <stdio.h>

void increment(int *num) {
    (*num)++;
}

int main() {
    int var = 10;
    printf("Antes da função: %d\n", var);
    increment(&var);
    printf("Depois da função: %d\n", var);

    return 0;
}
```

```
Antes da função: 10
Depois da função: 11


...Program finished with exit code 0
```

PONTEIROS

PREPARANDO O AMBIENTE

Para esta aula, utilizaremos 4 implementações, para o mesmo problema proposto.

O problema proposto:

Desenvolver um programa para somar dois valores.

As 4 implementações dizem respeitos aos seguintes paradigmas:

- Programação estruturada/linear
- Programação modular (uso de função)
- Programação estruturada/linear com uso de ponteiro
- Programação modular (uso de função) com uso de ponteiro

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#01 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

```
1   #include <stdio.h>
2  int main() {
3      int num = 10;
4      printf("Valor de num: %d\n", num);
5
6
7  int valor1=10;
8  int valor2=20;
9
10 printf("valor1: %d\n", valor1);
11 printf("valor2: %d\n", valor2);
12
13 int soma=valor1+valor2;
14   printf("==========RESULTADO============\n");
15   printf("soma: %d\n", soma);
16     return 0;
17 }
```

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#01 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

```c
  #include <stdio.h>
int main() {
    int num = 10;
    printf("Valor de num: %d\n", num);


int valor1=10;
int valor2=20;

printf("valor1: %d\n", valor1);
printf("valor2: %d\n", valor2);

int soma=valor1+valor2;
  printf("==========RESULTADO============\n");
  printf("soma: %d\n", soma);
    return 0;
}
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
				valor1										
				10										
402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#01 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

```
1   #include <stdio.h>
2  int main() {
3      int num = 10;
4      printf("Valor de num: %d\n", num);
5
6
7  int valor1=10;
8  int valor2=20;
9
10 printf("valor1: %d\n", valor1);
11 printf("valor2: %d\n", valor2);
12
13 int soma=valor1+valor2;
14   printf("==========RESULTADO============\n");
15   printf("soma: %d\n", soma);
16     return 0;
17 }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
				valor1										
				10										

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
	valor2													
	20													

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#01 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

```
1   #include <stdio.h>
2  int main() {
3      int num = 10;
4      printf("Valor de num: %d\n", num);
5
6
7  int valor1=10;
8  int valor2=20;
9
10 printf("valor1: %d\n", valor1);
11 printf("valor2: %d\n", valor2);
12
13 int soma=valor1+valor2;
14   printf("============RESULTADO============\n");
15   printf("soma: %d\n", soma);
16     return 0;
17 }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
				valor1										
				10										

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
	valor2													
	20													

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
							soma							
							30							

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#01 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

```
1   #include <stdio.h>
2  int main() {
3      int num = 10;
4      printf("Valor de num: %d\n", num);
5
6
7  int valor1=10;
8  int valor2=20;
9
10 printf("valor1: %d\n", valor1);
11 printf("valor2: %d\n", valor2);
12
13 int soma=valor1+valor2;
14   printf("==========RESULTADO============\n");
15   printf("soma: %d\n", soma);
16     return 0;
17 }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
				valor1										
				10										

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
	valor2													
	20													

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
							soma							
							30							

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#02 – PROGRAMAÇÃO MODULAR(USO DE FUNÇÃO)

```
1   int valor1=10;
2  int valor2=20;
3    #include <stdio.h>
4  /* fun   o para fazer a soma de 2 numeros*/
5  int somaValores(int parcela1, int parcela2) {
6
7    return parcela1+parcela2;
8  }
9    int main() {
10
11       printf("O resultado da soma de %d e %d    %d
            \n",valor1,valor2,somaValores(valor1,
            valor2));
12
13
14
15       return 0;
16   }
```

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#02 – PROGRAMAÇÃO MODULAR(USO DE FUNÇÃO)

```
int valor1=10;
int valor2=20;
  #include <stdio.h>
/* fun   o para fazer a soma de 2 numeros*/
int somaValores(int parcela1, int parcela2) {

  return parcela1+parcela2;
}
  int main() {

      printf("O resultado da soma de %d e %d     %d
         \n",valor1,valor2,somaValores(valor1,
         valor2));



      return 0;
  }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC
				valor1										
				10										

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#02 – PROGRAMAÇÃO MODULAR(USO DE FUNÇÃO)

```
int valor1=10;
int valor2=20;
#include <stdio.h>
/* fun  o para fazer a soma de 2 numeros*/
int somaValores(int parcela1, int parcela2) {

  return parcela1+parcela2;
}
  int main() {

      printf("O resultado da soma de %d e %d    %d
         \n",valor1,valor2,somaValores(valor1,
         valor2));



      return 0;
  }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
												valor2		
												20		

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC
				valor1										
				10										

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#02 – PROGRAMAÇÃO MODULAR(USO DE FUNÇÃO)

```
 int valor1=10;
int valor2=20;
  #include <stdio.h>
/* fun   o para fazer a soma de 2 numeros*/
int somaValores(int parcela1, int parcela2) {

  return parcela1+parcela2;
}
 int main() {

      printf("O resultado da soma de %d e %d     %d
         \n",valor1,valor2,somaValores(valor1,
         valor2));


      return 0;
 }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
												valor2		
												20		

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC
				valor1										
				10										

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#02 – PROGRAMAÇÃO MODULAR(USO DE FUNÇÃO)

```
 int valor1=10;
int valor2=20;
  #include <stdio.h>
/* fun   o para fazer a soma de 2 numeros*/
int somaValores(int parcela1, int parcela2) {

  return parcela1+parcela2;
}
  int main() {

      printf("O resultado da soma de %d e %d     %d
         \n",valor1,valor2,somaValores(valor1,
         valor2));


      return 0;
  }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
												valor2		
												20		

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
						parcela1								
						10								

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
											parcela2			
											20			

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC
				valor1										
				10										

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#02 – PROGRAMAÇÃO MODULAR(USO DE FUNÇÃO)

```
 int valor1=10;
int valor2=20;
  #include <stdio.h>
/* fun   o para fazer a soma de 2 numeros*/
int somaValores(int parcela1, int parcela2) {

  return parcela1+parcela2;
}
  int main() {

      printf("O resultado da soma de %d e %d     %d
         \n",valor1,valor2,somaValores(valor1,
         valor2));



      return 0;
  }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
												valor2		
												20		

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
						parcela1								
						10								

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
		return									parcela2			
		30									20			

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC
				valor1										
				10										

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#02 – PROGRAMAÇÃO MODULAR(USO DE FUNÇÃO)

```
int valor1=10;
int valor2=20;
  #include <stdio.h>
/* fun   o para fazer a soma de 2 numeros*/
int somaValores(int parcela1, int parcela2) {

  return parcela1+parcela2;
}
  int main() {

      printf("O resultado da soma de %d e %d    %d
         \n",valor1,valor2,somaValores(valor1,
         valor2));


      return 0;
  }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
												valor2		
												20		

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
						parcela1								
						10								

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
		return									parcela2			
		30									20			

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC
				valor1										
				10										

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#03 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

COM USO DE PONTEIRO

```
1  #include <stdio.h>
2  int main() {
3      int num = 10;
4      printf("Valor de num: %d\n", num);
5
6
7  int valor1=10;
8  int valor2=20;
9  int *ponteiroValor1=&valor1;
10 int *ponteiroValor2=&valor2;
11 printf("valor1: %d\n", *ponteiroValor1);
12 printf("valor2: %d\n", *ponteiroValor2);
13
14 int soma=*ponteiroValor1+*ponteiroValor2;
15   printf("==========RESULTADO============\n");
16   printf("soma: %d\n", soma);
```

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#03 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

COM USO DE PONTEIRO

```c
#include <stdio.h>
int main() {
    int num = 10;
    printf("Valor de num: %d\n", num);


int valor1=10;
int valor2=20;
int *ponteiroValor1=&valor1;
int *ponteiroValor2=&valor2;
printf("valor1: %d\n", *ponteiroValor1);
printf("valor2: %d\n", *ponteiroValor2);

int soma=*ponteiroValor1+*ponteiroValor2;
  printf("=========RESULTADO===========\n");
  printf("soma: %d\n", soma);
    return 0;
}
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
													valor1	
													10	

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#03 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

COM USO DE PONTEIRO

```c
#include <stdio.h>
int main() {
    int num = 10;
    printf("Valor de num: %d\n", num);


int valor1=10;
int valor2=20;
int *ponteiroValor1=&valor1;
int *ponteiroValor2=&valor2;
printf("valor1: %d\n", *ponteiroValor1);
printf("valor2: %d\n", *ponteiroValor2);

int soma=*ponteiroValor1+*ponteiroValor2;
  printf("=========RESULTADO===========\n");
  printf("soma: %d\n", soma);
    return 0;
}
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
	valor2													
	20													

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
													valor1	
													10	

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#03 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

COM USO DE PONTEIRO

```c
#include <stdio.h>
int main() {
    int num = 10;
    printf("Valor de num: %d\n", num);


int valor1=10;
int valor2=20;
int *ponteiroValor1=&valor1;
int *ponteiroValor2=&valor2;
printf("valor1: %d\n", *ponteiroValor1);
printf("valor2: %d\n", *ponteiroValor2);

int soma=*ponteiroValor1+*ponteiroValor2;
  printf("=========RESULTADO===========\n");
  printf("soma: %d\n", soma);
    return 0;
}
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
	valor2													
	20													

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
		*ponteiroValor1												
		409C												

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094		409C	40A0
													valor1	
			30										10	

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

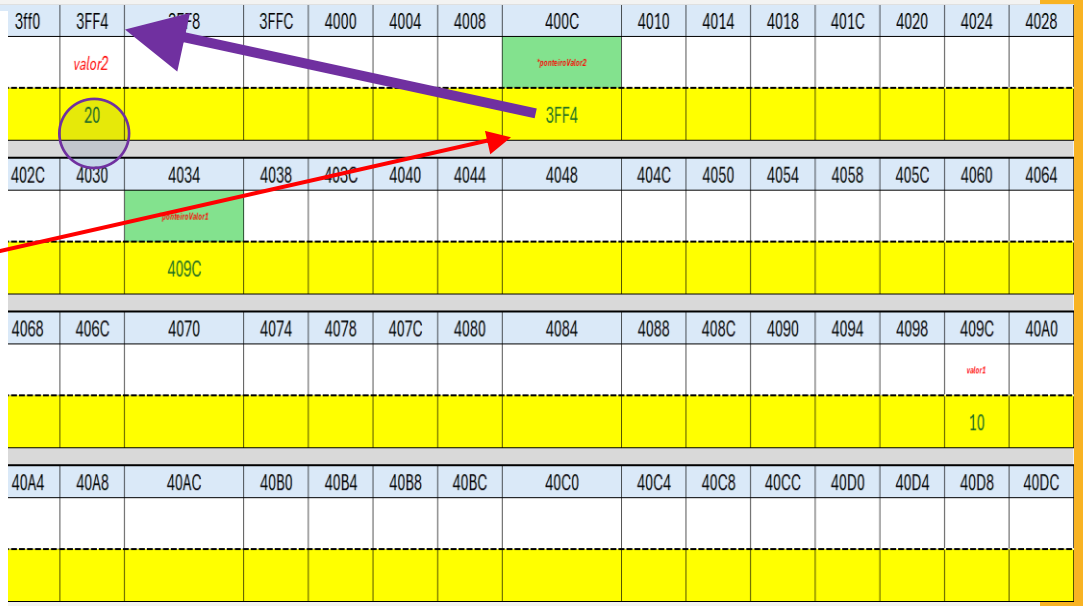
PROGRAMA#03 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

COM USO DE PONTEIRO

```c
#include <stdio.h>
int main() {
    int num = 10;
    printf("Valor de num: %d\n", num);


int valor1=10;
int valor2=20;
int *ponteiroValor1=&valor1;
int *ponteiroValor2=&valor2;
printf("valor1: %d\n", *ponteiroValor1);
printf("valor2: %d\n", *ponteiroValor2);

int soma=*ponteiroValor1+*ponteiroValor2;
  printf("=========RESULTADO===========\n");
  printf("soma: %d\n", soma);
    return 0;
}
```



PONTEIROS- REPRESENTAÇÃO MEMÓRIA

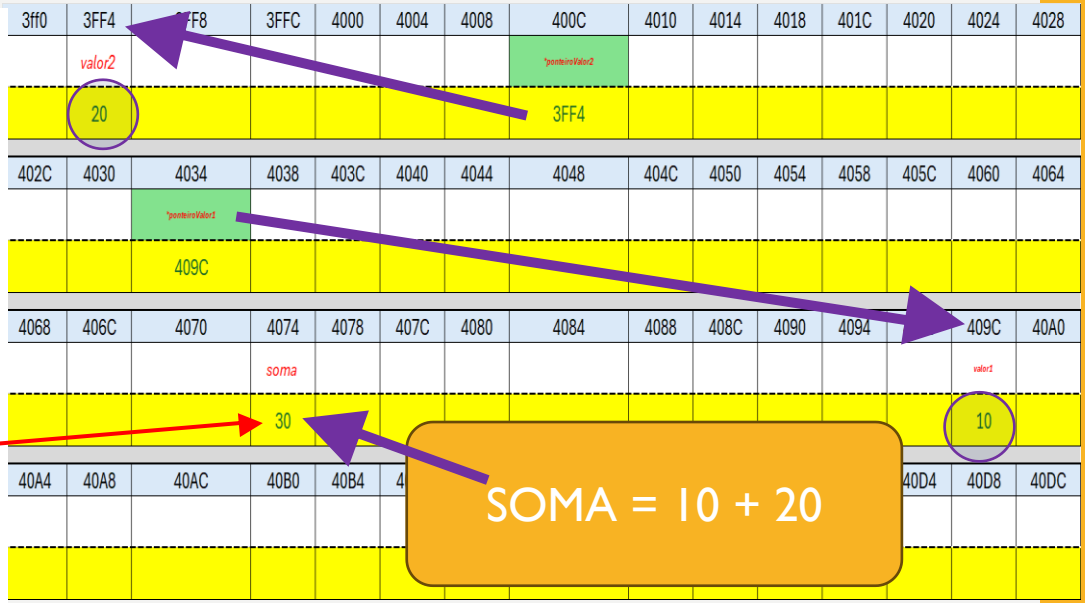
PROGRAMA#03 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

COM USO DE PONTEIRO

```c
#include <stdio.h>
int main() {
    int num = 10;
    printf("Valor de num: %d\n", num);


int valor1=10;
int valor2=20;
int *ponteiroValor1=&valor1;
int *ponteiroValor2=&valor2;
printf("valor1: %d\n", *ponteiroValor1);
printf("valor2: %d\n", *ponteiroValor2);

int soma=*ponteiroValor1+*ponteiroValor2;
  printf("==========RESULTADO===========\n");
  printf("soma: %d\n", soma);
    return 0;
}
```



PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#03 – PROGRAMAÇÃO ESTRUTURADA/LINEAR

COM USO DE PONTEIRO

```c
#include <stdio.h>
int main() {
    int num = 10;
    printf("Valor de num: %d\n", num);


int valor1=10;
int valor2=20;
int *ponteiroValor1=&valor1;
int *ponteiroValor2=&valor2;
printf("valor1: %d\n", *ponteiroValor1);
printf("valor2: %d\n", *ponteiroValor2);

int soma=*ponteiroValor1+*ponteiroValor2;
  printf("=========RESULTADO===========\n");
  printf("soma: %d\n", soma);
    return 0;
}
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
	valor2						*parcela2Ponteiro							
	20						3FF4							

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
		*parcela2Ponteiro												
		409C												

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
			return										valor1	
			30										10	

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#04 – PROGRAMAÇÃO MODULAR (USO DE FUNÇÃO)

COM USO DE PONTEIRO

```
1   #include <stdio.h>
2  int valor1=10;
3  int valor2=20;
4  int somaValores(int *parcela1Ponteiro, int *
     parcela2Ponteiro){
5    return *parcela1Ponteiro+*parcela2Ponteiro;
6  }
7  int main() {
8
9  printf("==========RESULTADO============\n");
10   printf("soma: %d\n",somaValores( &valor1, &
       valor2));
11     return 0;
12 }
```

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#04 – PROGRAMAÇÃO MODULAR (USO DE FUNÇÃO)

COM USO DE PONTEIRO

```
1   #include <stdio.h>
2 int valor1=10;
3 int valor2=20;
4 int somaValores(int *parcela1Ponteiro, int *
    parcela2Ponteiro){
5   return *parcela1Ponteiro+*parcela2Ponteiro;
6 }
7 int main() {
8
9 printf("==========RESULTADO============\n");
10   printf("soma: %d\n",somaValores( &valor1, &
       valor2));
11    return 0;
12 }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
			valor1											
			10											

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#04 – PROGRAMAÇÃO MODULAR (USO DE FUNÇÃO)

COM USO DE PONTEIRO

```
1   #include <stdio.h>
2 int valor1=10;
3 int valor2=20;
4 int somaValores(int *parcela1Ponteiro, int *
    parcela2Ponteiro){
5   return *parcela1Ponteiro+*parcela2Ponteiro;
6 }
7 int main() {
8
9 printf("==========RESULTADO============\n");
10  printf("soma: %d\n",somaValores( &valor1, &
      valor2));
11    return 0;
12 }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
			valor1											
			10											

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
valor2														
20														

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#04 – PROGRAMAÇÃO MODULAR (USO DE FUNÇÃO)

COM USO DE PONTEIRO

```
1   #include <stdio.h>
2  int valor1=10;
3  int valor2=20;
4  int somaValores(int *parcela1Ponteiro, int *
     parcela2Ponteiro){
5    return *parcela1Ponteiro+*parcela2Ponteiro;
6  }
7  int main() {
8
9  printf("==========RESULTADO===========\n");
10   printf("soma: %d\n",somaValores( &valor1, &
       valor2));
11     return 0;
12 }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
			valor1											
			10											

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
valor2														
20														

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0

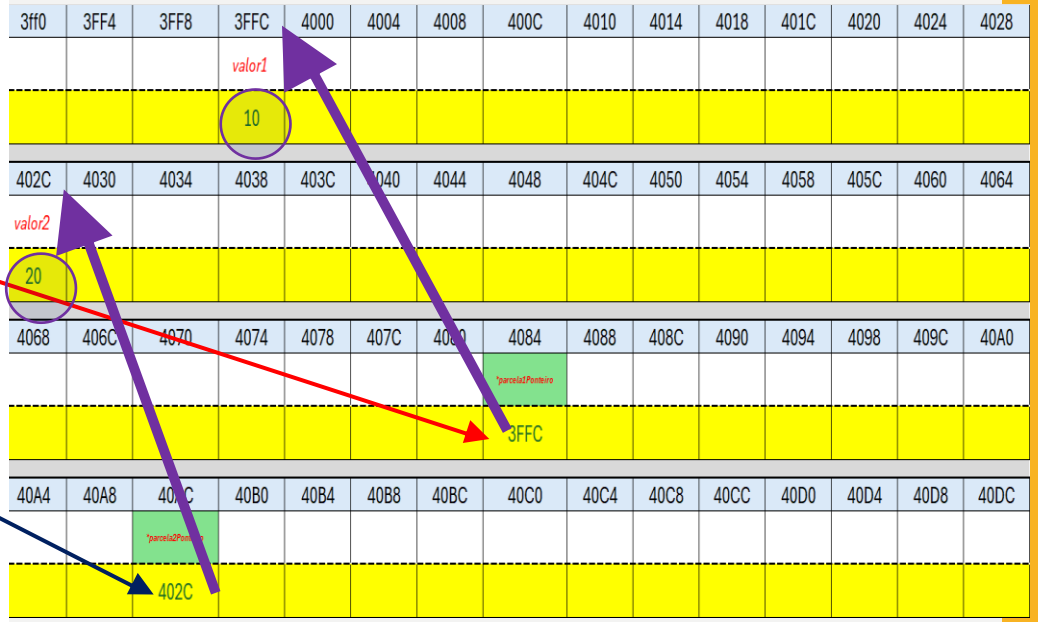
40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#04 – PROGRAMAÇÃO MODULAR (USO DE FUNÇÃO)

COM USO DE PONTEIRO

```
1   #include <stdio.h>
2  int valor1=10;
3  int valor2=20;
4  int somaValores(int *parcela1Ponteiro, int *
     parcela2Ponteiro){
5    return *parcela1Ponteiro+*parcela2Ponteiro;
6  }
7  int main() {
8
9  printf("==========RESULTADO==========\n");
10   printf("soma: %d\n",somaValores( &valor1, &
       valor2));
11   return 0;
12 }
```



PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#04 – PROGRAMAÇÃO MODULAR (USO DE FUNÇÃO)

COM USO DE PONTEIRO

```
1   #include <stdio.h>
2  int valor1=10;
3  int valor2=20;
4  int somaValores(int *parcela1Ponteiro, int *
     parcela2Ponteiro){
5    return *parcela1Ponteiro+*parcela2Ponteiro;
6  }
7  int main() {
8
9  printf("==========RESULTADO============\n");
10   printf("soma: %d\n",somaValores( &valor1, &
       valor2));
11    return 0;
12 }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
			valor1											
			10											

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
valor2														
20														

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
							*parcela1Ponteiro							
							3FFC							

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC
		*parcela2Ponteiro							return					
		402C							30					

PONTEIROS- REPRESENTAÇÃO MEMÓRIA

PROGRAMA#04 – PROGRAMAÇÃO MODULAR (USO DE FUNÇÃO)

COM USO DE PONTEIRO

```
1   #include <stdio.h>
2  int valor1=10;
3  int valor2=20;
4  int somaValores(int *parcela1Ponteiro, int *
     parcela2Ponteiro){
5    return *parcela1Ponteiro+*parcela2Ponteiro;
6  }
7  int main() {
8
9  printf("==========RESULTADO============\n");
10   printf("soma: %d\n",somaValores( &valor1, &
       valor2));
11   return 0;
12 }
```

3ff0	3FF4	3FF8	3FFC	4000	4004	4008	400C	4010	4014	4018	401C	4020	4024	4028
			valor1											
			10											

402C	4030	4034	4038	403C	4040	4044	4048	404C	4050	4054	4058	405C	4060	4064
valor2														
20														

4068	406C	4070	4074	4078	407C	4080	4084	4088	408C	4090	4094	4098	409C	40A0
							*parcela1Ponteiro							
							3FFC							

40A4	40A8	40AC	40B0	40B4	40B8	40BC	40C0	40C4	40C8	40CC	40D0	40D4	40D8	40DC
		*parcela2Ponteiro							return					
		402C							30					